
能力条件化通用路由

完整训练、新模型接入与部署流程

围绕主题段、候选 Fact、局部绝对质量标签与单段反事实替换，说明共享质量评分器的输入、输出、泛化边界和预算决策流程。

项目文档 · InfoBudget2

内容来源：上一轮完整技术分析

结论是：

这种方法可以训练出“结构上通用、统计上有条件通用”的路由器，但不能保证对任意新模型、任意新家族都天然可靠。

它的通用性来源于：

$$f_{\theta}(x_d, c_m) \rightarrow y_{d,m}$$

而不是：

$$x_d \rightarrow \{small, medium, large\}$$

其中模型能力 $\{c_m\}$ 必须作为每个候选模型的输入。它不是路由器要预测的类别标签，而是候选模型的条件特征。

一、这种路由器具有哪种“通用性”

通用性	是否具备	条件
候选模型数量可变	是，结构上支持	对每个 (主题段, 模型) 独立评分
模型顺序可变	是	共享评分函数，不使用固定输出位置
价格变化	是	价格不进入质量模型，只进入预算优化器
同家族新模型	有条件支持	新模型完成能力探针，且能力表示处于训练分布内
未见模型家族	有可能，但必须实验证明	多家族训练、leave-one-family-out 测试、OOD 检查
任意新模型完全零样本	不保证	没有行为探针就无法可靠了解新模型能力
任意新任务	不具备	当前能力指纹针对长期记忆 Fact 提取任务
任意 Retriever/Reader/Judge	不具备	更换下游组件会改变 QA 效用分布，需要重新校准

因此更准确的论文表述是：

这是一个面向长期记忆 Fact 提取任务的、能力条件化的动态模型池路由器。

而不是：

这是一个对任意模型和任意任务都通用的路由器。

二、为什么局部标签和 C 反事实标签必须同时存在

这两类标签解决的问题不同。

1. 局部 Fact 标签提供绝对刻度

例如：

$$y^{(fact)}_{d,m} = [\text{precision}, \text{recall}, \text{faithfulness}, \text{current_state}, \text{binding}, \text{stale_rejection}, \text{evidence_F1}]$$

它告诉路由器：

模型 $\{m\}$ 对主题段 $\{d\}$ 的实际提取质量是多少。

例如：

TEXT

seg_008 + qwen-small:
fact_quality = 0.52

seg_008 + qwen-medium:
fact_quality = 0.78

2. C 提供模型间的相对差异

例如：

TEXT

把 seg_008 从 small 替换为 medium
使相关 QA 正确率提高 0.15

它告诉路由器：

$$f(x_{d,c_medium}) - f(x_{d,c_small}) \approx 0.15$$

或者：

$$(x_{d,c_medium}) \setminus succ (x_{d,c_small})$$

3. 为什么不能只使用 C

如果只使用差值：

$$f(x_{d,c_medium}) - f(x_{d,c_small}) = 0.15$$

那么以下预测都满足条件：

TEXT

small = 0.10, medium = 0.25
small = 0.50, medium = 0.65
small = 0.80, medium = 0.95

它们排序相同，但绝对质量完全不同。

数学上，对所有模型分数同时加上任意常数 $k(x_d)$ ：

$$f'(x_{d,c_m}) = f(x_{d,c_m}) + k(x_d)$$

模型间差值不变。

所以：

- C 负责排序和下游边际增益；
- 局部 Fact 标签负责绝对质量和概率校准；
- 两者结合才能得到既能排序、又能估计质量的评分器。

三、是否包含“模型能力标签”

需要区分三个概念。

1. 模型能力表示：输入

$$c_m = \Psi(m)$$

它是模型在独立能力探针集上的表现，例如：

JSON

```
{
  "model_profile_id": "qwen-medium-extractor-v1",
  "model_family": "qwen",
  "memoryprint": {
    "fact_recall": 0.81,
    "target_precision": 0.77,
    "faithfulness": 0.91,
    "current_state": 0.74,
    "target_binding": 0.83,
    "stale_rejection": 0.70,
    "evidence_f1": 0.79
  },
  "conditional_fingerprint": {
    "temporal_update": 0.73,
    "multi_person": 0.82,
    "chinese": 0.88,
    "english": 0.79
  },
  "uncertainty": {
    "fact_recall_se": 0.03,
    "current_state_se": 0.06
  },
  "profile_mask": {
    "fact_recall": 1,
    "current_state": 1
  }
}
```

它要输入路由器。

2. 局部质量标签：训练目标

$$y^{(\text{fact})}_{d,m}$$

例如：

JSON

```
{
  "segment_id": "seg_008",
  "model_profile_id": "qwen-medium-extractor-v1",
  "local_quality": {
    "precision": 0.75,
    "recall": 0.80,
    "faithfulness": 1.0,
    "current_state": 1.0,
    "stale_rejection": 0.0,
    "evidence_f1": 0.67
  }
}
```

它是监督标签。

3. C 中的反事实标签：辅助训练目标

$$\Delta Q_{d,m_1,m_0}$$

例如：

JSON

```
{
  "segment_id": "seg_008",
  "baseline_model_profile_id": "qwen-small-extractor-v1",
  "candidate_model_profile_id": "qwen-medium-extractor-v1",
  "delta_quality": 0.15,
}
```

```
"preferred_model": "qwen-medium-extractor-v1",
"confidence": 0.82
}
```

它也是监督信号，但监督的是两个模型分数的差值和排序。

所以答案是：

使用路由时必须包含模型能力表示；但“模型能力”不是最终分类标签。路由器的训练标签是局部 Fact 质量、C 中的反事实质量差和模型偏好。

四、模型能力表示是如何生成的

能力向量不能通过模型名称硬编码，例如不能简单写：

```
TEXT
qwen-small -> [0, 0, 1]
qwen-medium -> [0, 1, 0]
qwen-large -> [1, 0, 0]
```

否则仍然只是模型 ID。

应使用独立能力探针集：

$$A = \{x_j, G_j, t_j\}_{j=1}^{N_A}$$

每个候选模型都在相同探针集上运行：

```
TEXT
候选模型
-> 固定 Fact 提取 Prompt
-> 固定 Schema
-> 能力探针集
-> Gold/Judge 评价
-> 聚合成 MemoryPrint
```

能力探针应覆盖：

- 简单事实；
- 多人物绑定；
- 指代消解；
- 时间表达；
- 知识更新；
- 旧值拒绝；
- 否定和取消；
- 中文、英文和中英混合；
- 跨句事实组合；
- 无记忆事实返回空集。

能力表示必须与最终训练和测试数据隔离，不能用最终测试集计算新模型的能力指纹，否则会产生泄漏。

项目文档已经明确将模型能力定义为条件输入，而不是训练目标，[模型能力条件化方案](#)。

五、训练阶段完整流程

流程定义

flowchart TD

```
A["训练数据：主题段、问题、答案、Evidence"] --> B["训练模型池注册"]
P["独立能力探针集"] --> C["生成每个模型的能力表示 c_m"]
B --> C
A --> D["每个模型处理同一批训练主题段"]
B --> D
D --> E["候选 Fact 集合 F(d,m)"]
E --> F["程序规则、Gold、固定 Judge、人工审计"]
F --> G["局部绝对质量标签 y_fact(d,m)"]
E --> H["构建基准与单段反事实 S"]
H --> I["Retriever、Reader、Judge"]
I --> J["C：质量差值 ΔQ 与模型偏好"]
C --> K["训练共享评分器 f(x,c)"]
G --> K
J --> K
K --> L["评分器 Checkpoint、Scaler、校准器、OOD 边界"]
```

阶段 0：冻结实验协议

输入

- Fact 提取 Prompt;
- 输出 Schema;
- Retriever;
- embedding 模型;
- Reader;
- Judge;
- 数据划分;
- 质量指标定义。

输出

JSON

```
{
  "protocol_version": "capability_router_v1",
  "extraction_prompt_hash": "...",
  "retriever_hash": "...",
  "reader_prompt_hash": "...",
  "judge_prompt_hash": "...",
  "split_manifest_hash": "..."
}
```

这些组件在标签生成期间不能随意变化，否则前后标签不在同一尺度上。

阶段 1：生成模型能力表示

输入

对每个训练模型 m ：

TEXT

模型配置

+ 固定提取 Prompt

- + 独立能力探针集
- + Gold/固定 Judge

输出

TEXT
model_capability_profiles.json

每个模型对应：

```
c_m= [ MemoryPrint, conditional\_fingerprint, static\_configuration, uncertainty, mask ]
```

这里可以保留：

- 模型 ID，用于审计；
- 模型家族，用于分组评估；
- 参数量和量化方式，用于弱先验；
- 但不建议把模型 ID 作为可学习的类别特征。

阶段 2：生成所有候选 Fact

对所有训练主题段和训练模型：

```
F_d,m=h_m(x_d)
```

输入

JSON
{
 "segment_id": "seg_008",
 "segment_text": "...",
 "candidate_model_id": "qwen-medium",
 "extraction_prompt_version": "..."
}

输出

JSON
{
 "segment_id": "seg_008",
 "model_profile_id": "qwen-medium-extractor-v1",
 "facts": [
 {
 "fact_id": "f1",
 "fact_text": "...",
 "source_ids": [4, 7]
 }
],
 "token_usage": {},
 "cost_snapshot": {}
}

训练时需要让多个模型尽量处理同一批主题段，才能公平比较模型差异。

阶段 3：生成局部绝对质量标签

输入

- 原始主题段；

- 候选 Fact;
- Gold/Silver reference facts;
- Gold evidence;
- 固定 Fact Judge;
- 程序规则。

输出

TEXT

local_fact_quality_labels.jsonl

一行对应一个 (segment, model):

(x_{d,c,m}, y^{^(fact)}_{d,m})

例如:

JSON

```
{
  "segment_id": "seg_008",
  "model_profile_id": "qwen-medium-extractor-v1",
  "labels": {
    "precision": 0.75,
    "recall": 0.80,
    "faithfulness": 1.0,
    "current_state": 1.0,
    "target_binding": 1.0,
    "stale_rejection": 0.0,
    "evidence_f1": 0.67
  },
  "label_mask": {
    "precision": 1,
    "recall": 1,
    "faithfulness": 1,
    "current_state": 1,
    "target_binding": 1,
    "stale_rejection": 1,
    "evidence_f1": 1
  },
  "label_confidence": {},
  "label_sources": {}
}
```

阶段 4：生成 C 反事实标签

输入

- 当前基准路由;
- 冻结 L/M/H 候选库;
- 主题段相关问题集合 \(\mathcal{Q}_d\);
- 固定 Retriever、Reader 和 Judge。

操作

只替换一个主题段的模型:

$m_0 \rightarrow m_1$

输出

TEXT

counterfactual_effects.jsonl

包括：

$$\Delta Q_{d,m_1,m_0}$$

以及：

$$m_1 \backslash \text{succ } m_0$$

或：

$$m_1 \approx m_0$$

阶段 5：组装训练样本

最终至少有三种训练记录。

点式绝对质量样本

JSON

```
{
  "segment": "x_d",
  "candidate_model_profile": "c_m",
  "target_local_quality": "y_fact_d_m"
}
```

成对差值样本

JSON

```
{
  "segment": "x_d",
  "model_profile_a": "c_m0",
  "model_profile_b": "c_m1",
  "target_delta_quality": 0.15
}
```

问题级组合样本

JSON

```
{
  "question_id": "q1",
  "evidence_segment_ids": ["d1", "d2"],
  "route": {
    "d1": "model_a",
    "d2": "model_b"
  },
  "judge_correct": true
}
```

阶段 6：训练共享评分器

输入编码：

$$h_d = E_x(x_d)$$

$$e_m = E_c(c_m)$$

共享评分器：

$$y_{d,m} = f_{\theta}(h_{d,e_m})$$

推荐同时输出：

JSON

```
{
  "predicted_quality": {
    "precision": 0.77,
    "recall": 0.73,
    "faithfulness": 0.91,
    "current_state": 0.84,
    "target_binding": 0.88,
    "stale_rejection": 0.76,
    "evidence_f1": 0.79
  },
  "predicted_uncertainty": {
    "precision": 0.04,
    "recall": 0.07
  }
}
```

综合质量可以通过预先声明的权重计算：

$$q_{d,m} = \sum_k w_k(x_d) y_{d,m,k}$$

训练损失：

$$L = L_{\text{local-fact}} + \eta L_{\Delta} + \gamma L_{\text{rank}} + \beta L_{\text{QA}} + \rho L_{\text{calibration}}$$

价格不进入该损失。

阶段 7：模型级泛化验证

必须至少进行：

- **leave-one-model-out**;
- **leave-one-family-out**;
- 新模型 probe 数量消融;
- 无能力向量消融;
- 只有模型静态属性消融;
- 只有七维 MemoryPrint;
- MemoryPrint + 条件行为指纹;
- OOD 校准;
- 相对 Oracle regret;
- quality-cost frontier。

如果只在 Qwen-small/medium/large 上训练和测试，只能证明固定 Qwen 池有效，不能证明跨家族通用。

六、新模型接入流程

训练完成后，评分器冻结。

假设出现一个训练时未见的 Llama 模型。

流程定义

flowchart TD

```
A["新模型及部署配置"] --> B["运行独立能力探针"]
B --> C["生成能力表示 c_new"]
C --> D["与训练能力分布比较"]
D --> |分布内| E["注册到候选模型池"]
D --> |轻度 OOD| F["增加探针或少量校准"]
D --> |严重 OOD| G["保守路由或周期性重训"]
E --> H["参与主题段逐候选评分"]
F --> H
G --> H
```

输入

JSON

```
{
  "model_id": "llama-new",
  "model_family": "llama",
  "prompt_version": "...",
  "chat_template_hash": "...",
  "quantization": "bf16",
  "decoding": {
    "temperature": 0.0
  },
  "probe_set_version": "memory_apititude_v1"
}
```

输出

JSON

```
{
  "model_profile_id": "llama-new-extractor-v1",
  "capability_vector": [],
  "conditional_fingerprint": {},
  "uncertainty": {},
  "ood_score": 0.18,
  "registration_status": "accepted"
}
```

如果 OOD 分数过高：

JSON

```
{
  "registration_status": "calibration_required"
}
```

此时不能强行宣称零样本可靠。

项目设计也要求新模型运行能力探针、生成能力表示并检查 OOD，[模型能力条件化方案](#)。

七、部署时使用路由器的完整流程

训练期会预生成所有模型候选

Fact，以便构造监督；真实部署期不能先让所有模型都提取一遍，否则路由失去节省成本的意义。

真实部署流程如下。

流程定义

flowchart TD

```
A["新对话/历史 H"] --> B["主题分割"]
B --> C["主题段 x1...xN"]
M["当前候选模型池及能力表示 c1...cM"] --> D["共享质量评分器"]
C --> D
```

```
D --> E["质量矩阵 Q: N×M"]
C --> F["Token/成本预测"]
M --> F
F --> G["成本矩阵 C: N×M"]
E --> H["预算优化器"]
G --> H
BUD["写入预算 B"] --> H
H --> I["主题段→具体模型ID映射 D"]
I --> J["只调用选中的模型提取 Fact"]
J --> K["组装并持久化 S"]
K --> L["后续问题从 S 检索并复用"]
```

第一步：主题分割

输入

JSON

```
{
  "sample_id": "sample_new",
  "conversation_history": [...]
}
```

输出

JSON

```
{
  "segments": [
    {
      "segment_id": "seg_001",
      "segment_text": "...",
      "token_count": 620,
      "turn_ids": [1, 2, 3]
    }
  ]
}
```

第二步：加载候选模型池

输入

当前允许使用的模型：

JSON

```
{
  "candidate_models": [
    {
      "model_profile_id": "qwen-small-extractor-v1",
      "capability_vector": [],
      "price": {}
    },
    {
      "model_profile_id": "llama-new-extractor-v1",
      "capability_vector": [],
      "price": {}
    }
  ]
}
```

候选池可以有两个、三个、五个或者更多模型。

第三步：构造所有主题段—模型 pair

假设有 $\backslash(N)$ 个主题段、 $\backslash(M)$ 个候选模型：

$$P = \backslash(x_{d,c_m}):d=1,\ldots,N;m=1,\ldots,M\backslash$$

路由质量模型的输入

JSON

```
{
  "segment": {
    "segment_id": "seg_001",
    "text_embedding": [],
    "numeric_features": {
      "token_count": 620,
      "turn_count": 3,
      "entity_density": 0.12,
      "temporal_signal": 1,
      "update_signal": 0
    }
  },
  "candidate_model": {
    "model_profile_id": "llama-new-extractor-v1",
    "capability_vector": [],
    "conditional_fingerprint": {},
    "uncertainty": {},
    "profile_mask": {}
  }
}
```

这里包含模型能力表示，但不包含：

- 当前模型价格；
- 训练时的 small/medium/large 标签；
- 问题答案；
- Judge 标签；
- 测试 evidence。

第四步：质量评分器输出

对于每个 pair:

$$(x_{d,c_m}) \rightarrow (y_{d,m}, \sigma_{d,m})$$

例如：

JSON

```
{
  "segment_id": "seg_001",
  "model_profile_id": "llama-new-extractor-v1",
  "predicted_quality": {
    "precision": 0.78,
    "recall": 0.82,
    "faithfulness": 0.93,
    "current_state": 0.71,
    "target_binding": 0.85,
    "stale_rejection": 0.69,
    "evidence_f1": 0.76
  },
  "overall_quality": 0.81,
}
```

```
"quality_uncertainty": 0.06,
"quality_lcb": 0.75
}
```

全部 pair 构成质量矩阵：

$$Q = \begin{bmatrix} q_{1,1} & q_{1,2} & \dots & q_{1,M} \\ q_{2,1} & q_{2,2} & \dots & q_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ q_{N,1} & q_{N,2} & \dots & q_{N,M} \end{bmatrix}$$

项目设计中也区分了质量预测器输出和最终路由器输出，模型能力条件化方案。

第五步：预测成本

成本模型输入：

TEXT

主题段 token/结构
+ 候选模型 tokenizer
+ 预计输出长度
+ 当前价格表

输出：

JSON

```
{
  "segment_id": "seg_001",
  "model_profile_id": "llama-new-extractor-v1",
  "predicted_input_tokens": 850,
  "predicted_output_tokens": 210,
  "predicted_cost": 0.0034
}
```

全部候选构成成本矩阵：

$$C \in \mathbb{R}^{(N \times M)}$$

价格可以随时更新，不需要重训质量预测器。

第六步：预算优化

输入：

JSON

```
{
  "quality_matrix": "...",
  "cost_matrix": "...",
  "write_budget": 0.5,
  "risk_coefficient": 1.0
}
```

优化目标：

$$\max_{a_1, \dots, a_N} \sum_d \leq \text{ft} (q_{d,a_d} - \kappa \sigma_{d,a_d})$$

约束：

$$\sum_d C_{d,a_d} \leq B$$

输出：

JSON

```
{
```

```
"sample_id": "sample_new",
"budget": 0.5,
"predicted_total_cost": 0.43,
"predicted_total_quality": 0.84,
"route": [
  {
    "segment_id": "seg_001",
    "selected_model_profile_id": "qwen-small-extractor-v1",
    "predicted_quality": 0.79,
    "predicted_cost": 0.0012
  },
  {
    "segment_id": "seg_002",
    "selected_model_profile_id": "llama-new-extractor-v1",
    "predicted_quality": 0.91,
    "predicted_cost": 0.0043
  }
]
}
```

这里输出的是具体模型 ID，不是抽象的 small/medium/large。

第七步：执行实际 Fact 提取

只调用每个主题段被选中的模型：

TEXT

```
seg_001 -> qwen-small
seg_002 -> llama-new
seg_003 -> qwen-medium
```

输入

JSON

```
{
  "segment_id": "seg_002",
  "segment_text": "...",
  "selected_model_id": "llama-new",
  "extraction_prompt": "..."
}
```

输出

JSON

```
{
  "segment_id": "seg_002",
  "extractor_model": "llama-new",
  "facts": [
    {
      "fact_id": "...",
      "fact_text": "...",
      "source_ids": [7, 8]
    }
  ],
  "actual_tokens": {},
  "actual_cost": 0.0041
}
```

第八步：组装并持久化 S

将被选模型产生的 Fact 写入 S：

JSON

```
{
  "assembly_id": "...",
  "sample_id": "sample_new",
  "route": {
    "seg_001": "qwen-small-extractor-v1",
    "seg_002": "llama-new-extractor-v1"
  },
  "point_count": 37,
  "status": "ready"
}
```

每个主题段在一个 assembly 中只对应一个模型。

第九步：后续 QA 复用

后续问题到来时：

TEXT

Question

- > 从固定 S 检索 Top-K Fact
- > Reader 回答
- > 可选 Judge/反馈

不再重新路由由历史主题段，也不重新调用所有候选提取模型。

第十步：部署监控

记录：

- 预测成本与实际成本差；
- 预测质量和后续 QA 表现；
- 新模型 OOD 分数；
- 模型能力漂移；
- 模型版本、Prompt 或量化方式变化；
- 路由选择频率；
- 大模型集中率；
- 预算违反率。

这些新数据可以进入下一周期训练，但不能在线直接修改正式路由器，除非采用经过验证的在线学习协议。

八、训练与部署的输入输出汇总

阶段	输入	输出
能力指纹生成	模型 + 独立探针 + Gold/Judge	模型能力表示 $\backslash(c_m\backslash)$
候选提取	主题段 $\backslash(x_d\backslash)$ + 模型 $\backslash(m\backslash)$	候选 Fact $\backslash(F_{d,m}\backslash)$
局部质量评价	Fact + Gold/Silver/Judge	绝对标签 $\backslash(y^{\{fact\}}_{d,m}\backslash)$
C 反事实评价	基准 S + 单段替换 S + 问题	$\backslash(\Delta Q\backslash)$ 、偏好、置信度
评分器训练	$\backslash(x_d, c_m, y^{\{fact\}}, \Delta Q\backslash)$	质量评分器 checkpoint
新模型接入	新模型 + 独立探针	$\backslash(c_{new}\backslash)$ 、OOD 状态
部署质量预测	$\backslash(x_d + c_m\backslash)$	$\backslash(\hat{q}_{d,m}, \hat{\sigma}_{d,m}\backslash)$
成本预测	主题段 + tokenizer + 价格	$\backslash(C_{d,m}\backslash)$
预算优化	质量矩阵 + 成本矩阵 + 预算	主题段→具体模型 ID
实际提取	主题段 + 被选模型	Fact
持久化	Fact + route provenance	S assembly

九、最终回答

这种路由器可以具有通用性质，但必须满足以下条件：

1. 路由输入是“主题段 + 单个候选模型能力表示”；
2. 同一个共享评分器评价所有候选模型；
3. 训练标签是局部 Fact 质量和 C 中的反事实差值/偏好；
4. 候选模型数量不固定；
5. 模型价格与质量预测解耦；
6. 新模型先运行独立能力探针；
7. 对新模型进行 OOD 检查；
8. 通过 leave-one-model-out 和 leave-one-family-out 实验证明泛化；
9. 能力探针、训练数据和最终测试集严格隔离；
10. 严重 OOD 的新模型需要少量校准或重新训练。

模型能力表示在全流程中的位置是：

主题段 x_d + 候选模型能力 c_m → 该模型的预测质量 $y_{d,m}$

它包含模型能力信息，但不把能力当作最终类别标签。最终路由输出也不是 **small/medium/large**，而是：

$segment_id \rightarrow selected\ concrete\ model_profile_id$

这才是能够支持动态候选模型池和跨家族接入的完整形式。